

SOOKMYUNG LIKELION 13TH FE 1주차 세미나 - JS 기초

숙명 멋대 13기 FE 세미나

2025.04.08(TUE) 19:00

목차

파트별 세미나 1주차 - JS 기초

01

자바스크립트와 콘솔

02

변수 선언

03

데이터 타입

04

연산자 및 조건문

05

반복문

06

함수 선언 및 호출

07

JS 기초 개념 실습

08

과제 안내

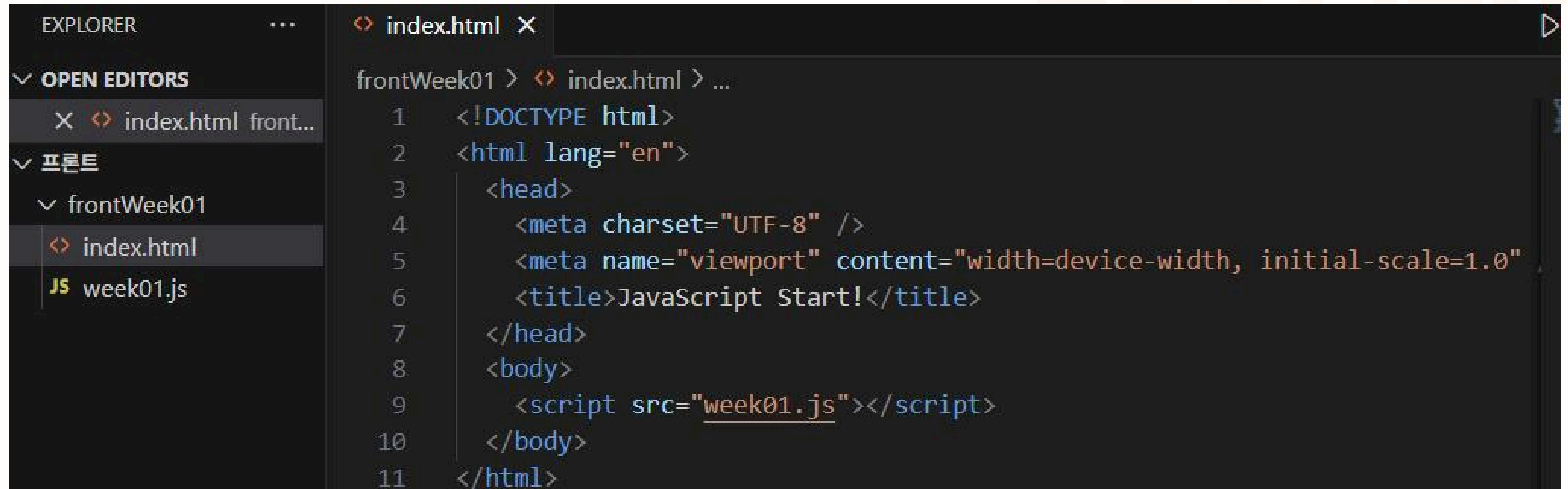
01

자바스크립트와 콘솔

자바스크립트와 콘솔

HTML, JS SETTINGS

POSSIBILITY
TO
REALLY



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar is open, showing a project structure with a folder named 'frontWeek01' containing 'index.html' and 'week01.js'. The main editor area displays the content of 'index.html'. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>JavaScript Start!</title>
7   </head>
8   <body>
9     <script src="week01.js"></script>
10  </body>
11 </html>
```

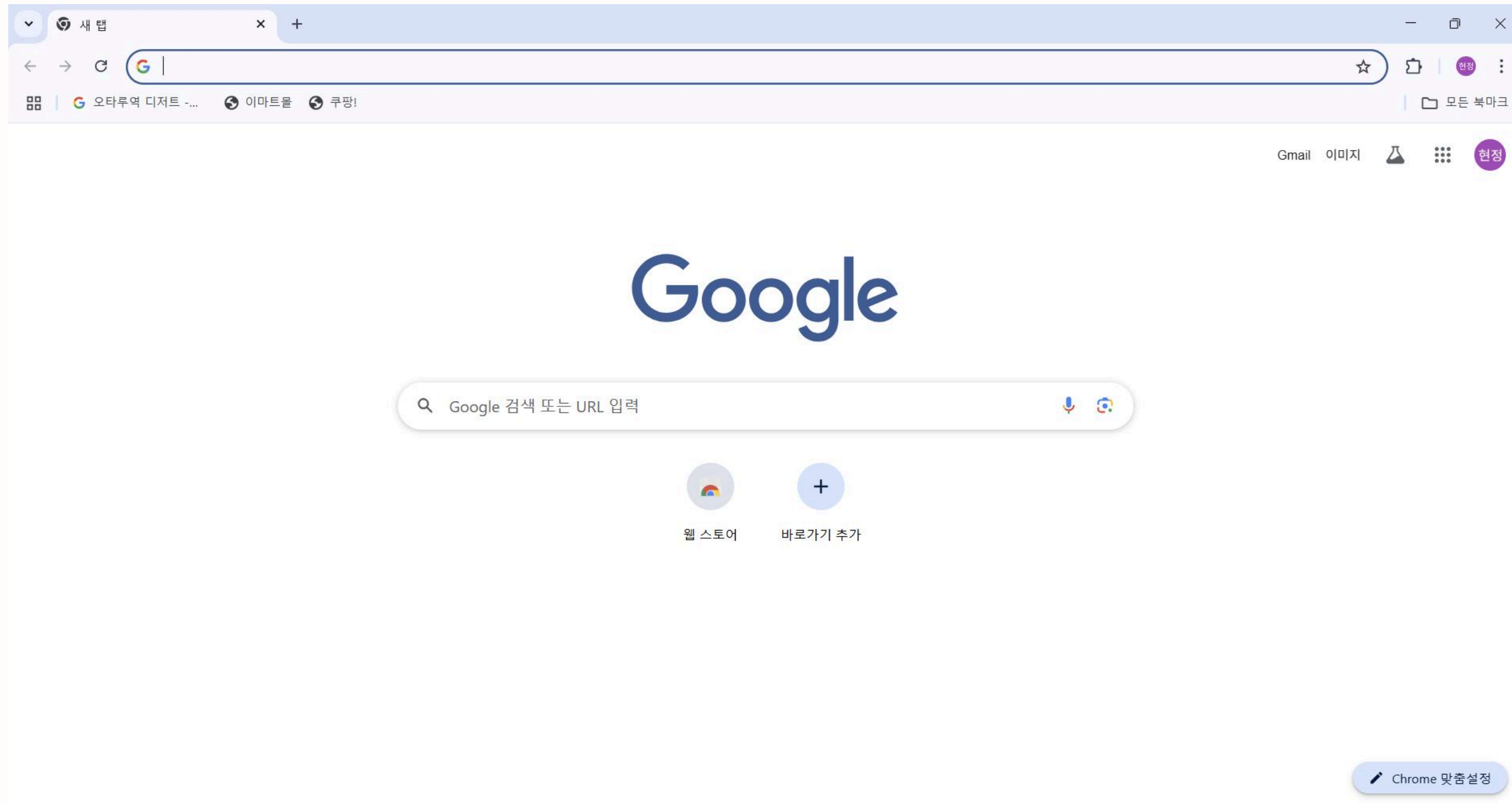

자바스크립트와 콘솔 콘솔이란?

POSSIBILITY
TO
REALITY

로딩된 웹페이지의 **에러를 확인**하거나
자바스크립트 소스코드에 작성한 **console.log** 메서드의 **실행 결과**를 확인할 수 있는
개발자 도구의 기능

자바스크립트와 콘솔

개발자 도구 사용하기



POSSIBILITY
TO
REALITY

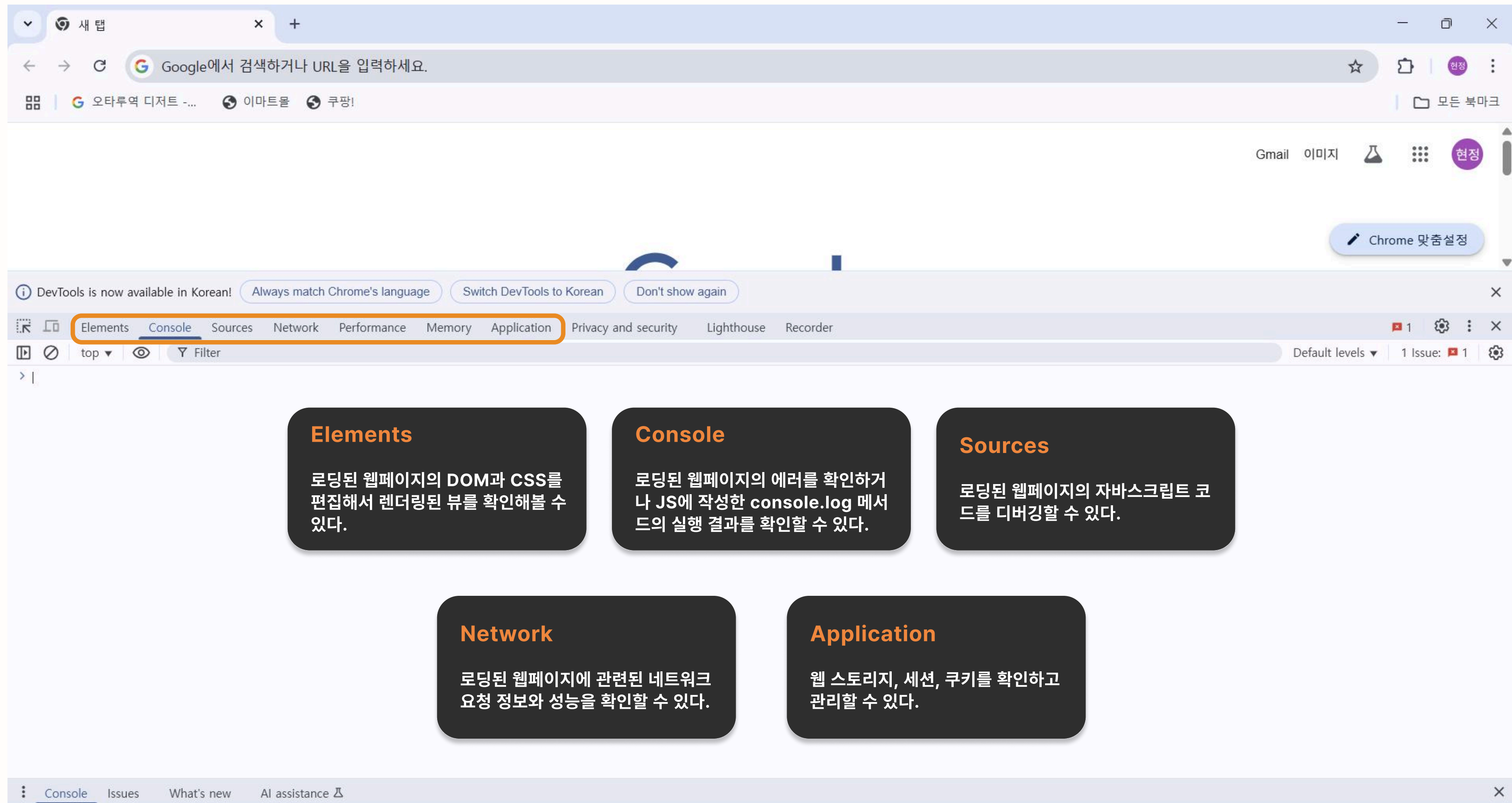
1. Chrome창 열기

2. Windows - **F12**

macOS - **Command + Option + I**

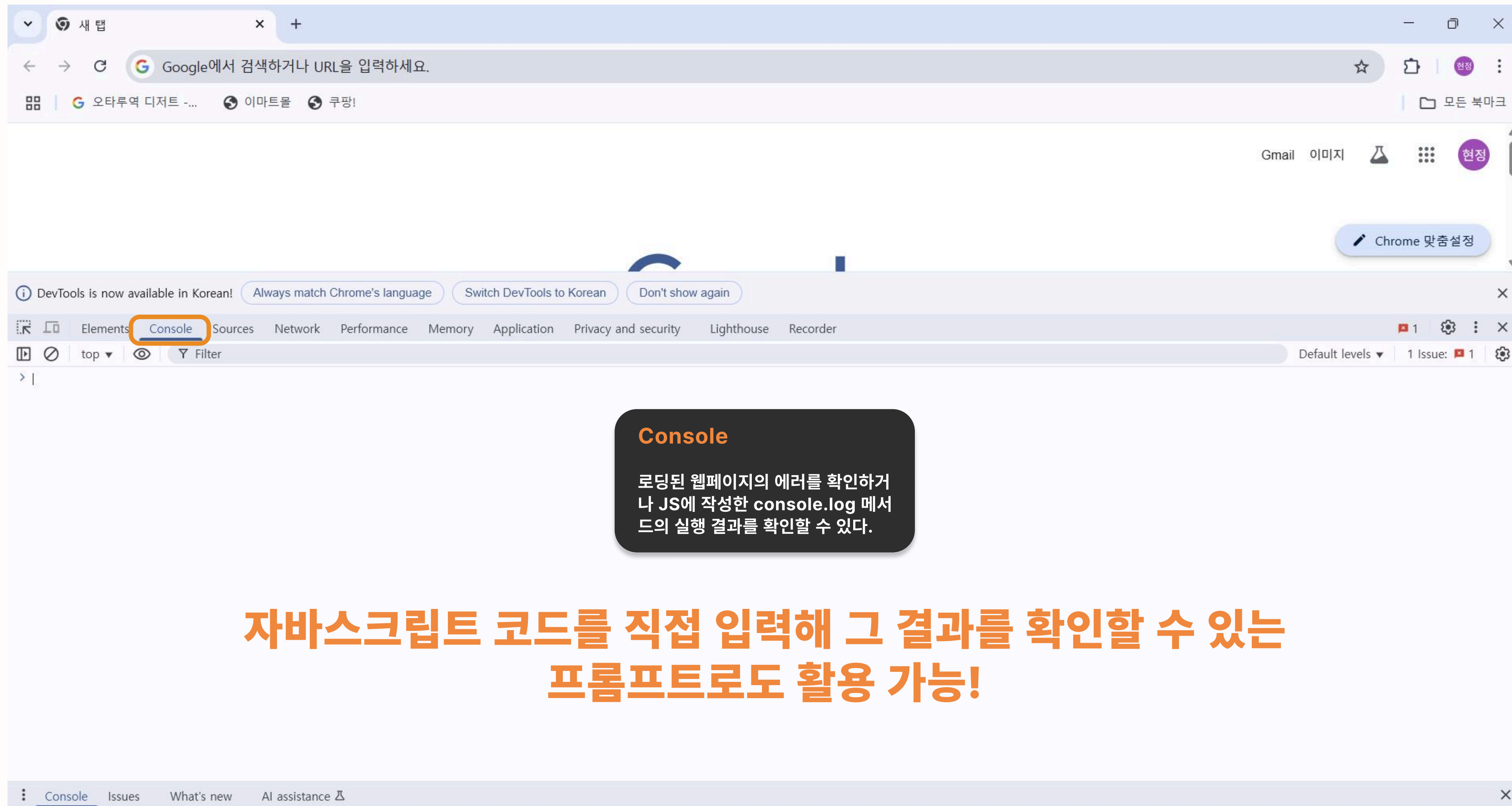
자바스크립트와 콘솔

개발자 도구 사용하기



자바스크립트와 콘솔

개발자 도구 사용하기



02

변수 선언

변수 선언

변수란?

“하나의 값을 저장하기 위해 확보한 메모리 공간”

프로그래밍 언어는 기억하고 싶은 값을 메모리에 저장하고,
저장된 값을 읽어 들여 재사용하기 위해 변수 메커니즘을 사용한다.

변수 이름: 메모리 공간에 저장된 값을 식별할 수 있는 고유한 이름

변수 값: 변수에 저장된 값

할당: 변수에 값을 저장하는 것

참조: 변수에 저장된 값을 읽어 들이는 것

POSSIBILITY
TO
REALITY

변수 선언

변수 선언하기

변수를 사용하려면 반드시 선언이 필요하다!

변수를 선언할 때는 **var, let, const** 키워드를 사용한다.

var

변수에 바뀔 수 있는 값을 넣고 싶을 때

POSSIBILITY
TO
REALITY

```
1  var a = 5;  
2  var a = 7;  
3  
4  console.log(a);
```

변수 재선언, 변수 재할당 가능

변수 선언

변수 선언하기

변수를 사용하려면 반드시 선언이 필요하다!

변수를 선언할 때는 **var, let, const** 키워드를 사용한다.

let

변수에 바뀔 수 있는 값을 넣고 싶을 때

```
1 let a = 5;  
2 let a = 7;  
3  
4 console.log(a);
```

```
1 let a = 5;  
2 a = 7;  
3  
4 console.log(a);
```

변수 재선언 불가능 / 변수 재할당 가능

POSSIBILITY
TO
REALITY

변수 선언

변수 선언하기

변수를 사용하려면 반드시 선언이 필요하다!

변수를 선언할 때는 **var, let, const** 키워드를 사용한다.

var

let

var 키워드는 함수 레벨 스코프!
함수가 아닌 코드 블록 내 변수는 전역 변수 처리

변수 선언

변수 선언하기

변수를 사용하려면 반드시 선언이 필요하다!

변수를 선언할 때는 **var, let, const** 키워드를 사용한다.

const

변수에 바뀔 수 없는 상수를 넣고 싶을 때

```
1  const a = 5;  
2  const myName = "hyeonjeong";
```

변수 재선언 불가능 / 변수 재할당 불가능

POSSIBILITY
TO
REALITY

변수 선언

변수 호이스팅

JS week01.js X

frontWeek01 > JS week01.js > ...

1 console.log(score);

2

3 var score;

POSSIBILITY
TO
REALITY

🤔 결과를 예상해보세요!! 🤔

변수 선언

변수 호이스팅

POSSIBILITY
TO
REALITY

자바스크립트는

변수 선언을 런타임 시점이 아니라

그 이전 단계에서 먼저 실행하여 undefined로 초기화한다.

let, const를 사용했을 경우도 호이스팅이 적용되나,
명확한 에러 표시가 난다.

```
JS week01.js X
frontWeek01 > JS week01.js > ...
1 console.log(score);
2
3 var score;
```



03

데이터 타입

데이터 타입

숫자 타입 / 불리언 타입 / 문자열 타입

자바스크립트의 숫자 타입

모든 수를 실수로 처리하며, 정수만 표현하기 위한 데이터 타입이 별도로 존재하지 않는다.

자바스크립트의 불리언 타입

true / false

자바스크립트의 문자열 타입

텍스트 데이터를 나타내는 데 사용하며, 작은따옴표, 큰따옴표, 또는 백틱으로 텍스트를 감싼다.

POSSIBILITY
TO
REALITY

데이터 타입

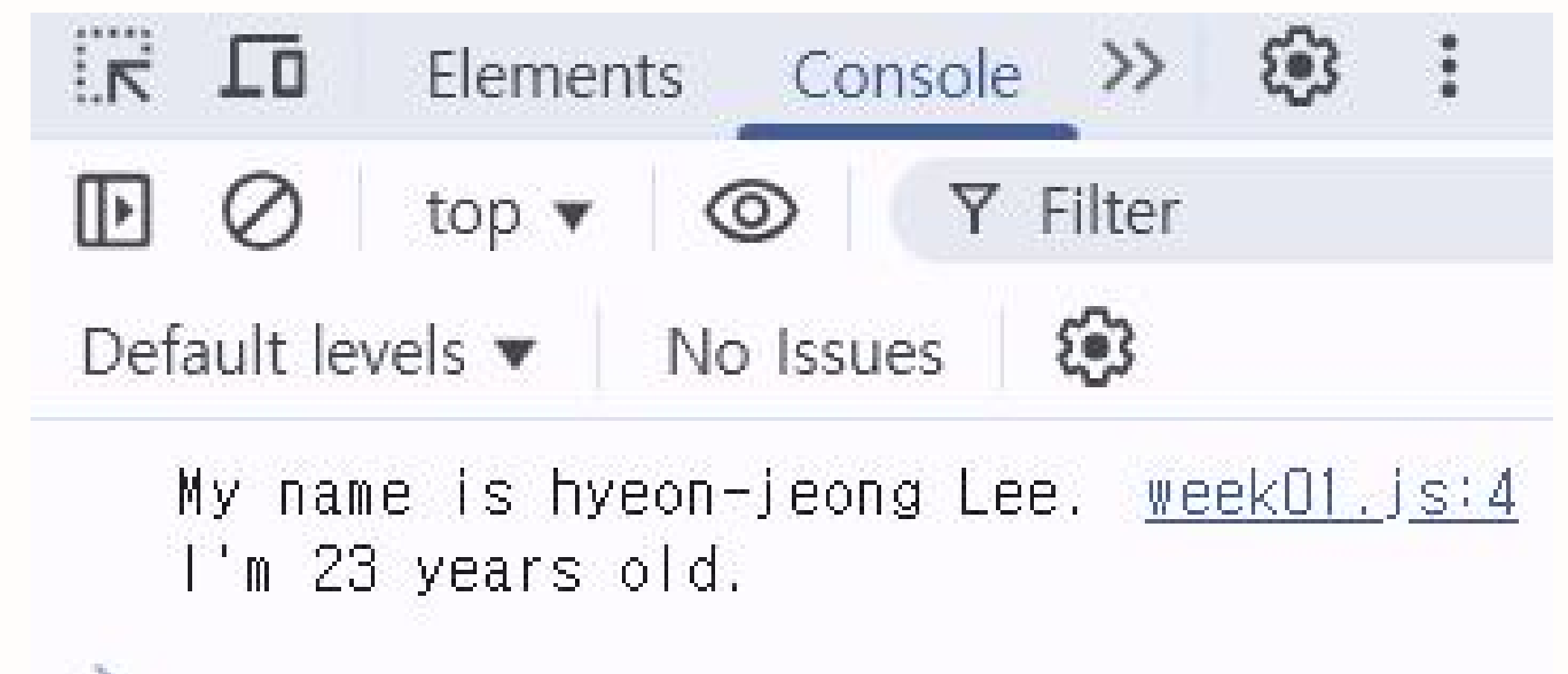
템플릿 리터럴

“백틱을 사용해 표현하는 문자열”

템플릿 리터럴을 사용해 문자열을 표현하면,
줄바꿈이나 공백을 그대로 표현하는 **멀티라인 문자열**을 작성할 수 있고,
표현식을 삽입하여 문자열을 나타낼 수 있다.

POSSIBILITY
TO
REALITY

```
JS week01.js X
frontWeek01 > JS week01.js > ...
1 let first = "hyeon-jeong";
2 let last = "Lee";
3
4 console.log(`My name is ${first} ${last}.
5 I'm ${20 + 3} years old.`);
```



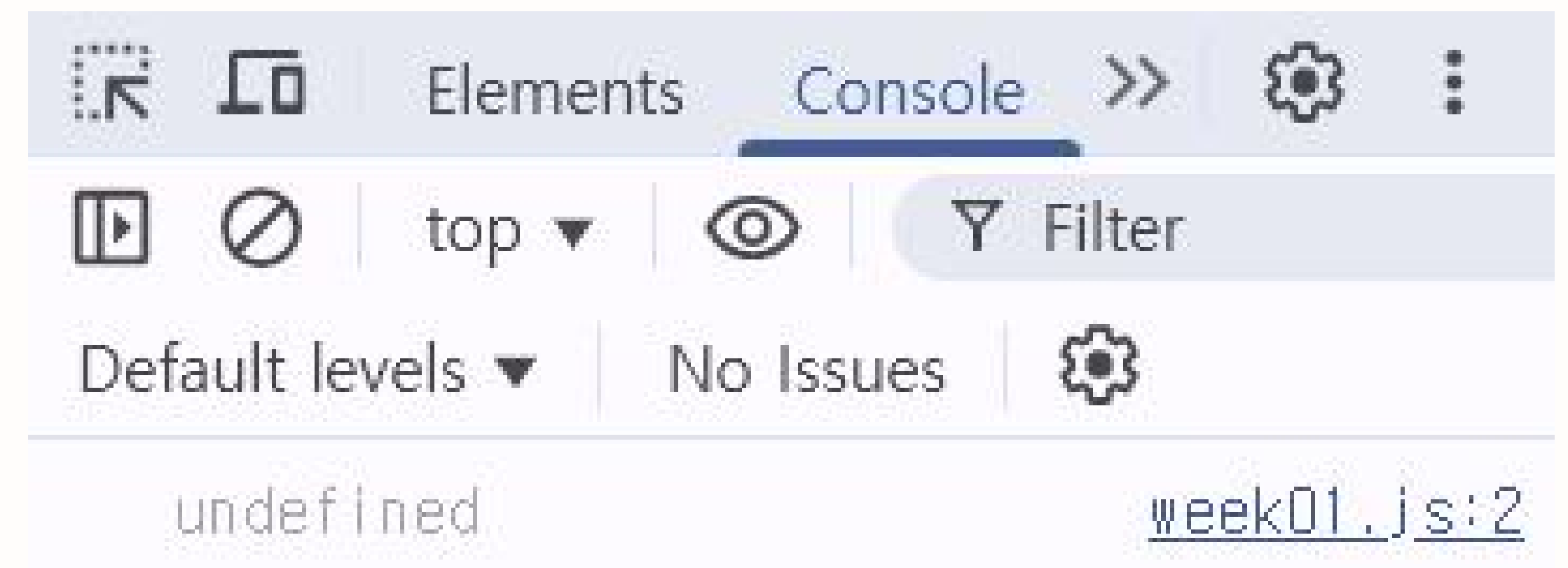
데이터 타입

undefined 타입

변수 선언에 의해 확보된 메모리에 할당이 이루어지지 않았을 경우

자바스크립트는 변수를 선언하는 시점에 undefined 값으로 초기화한다.
변수를 선언한 이후 값을 할당하지 않은 변수를 참조하면 undefined가 반환된다.

```
JS week01.js X
frontWeek01 > JS week01.js > ...
1  let likelion;
2  console.log(likelion);
```



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the value 'undefined' on the left and the source file and line number 'week01.js:2' on the right. The console interface includes various icons for navigation and filtering, and the text 'No Issues' is visible.

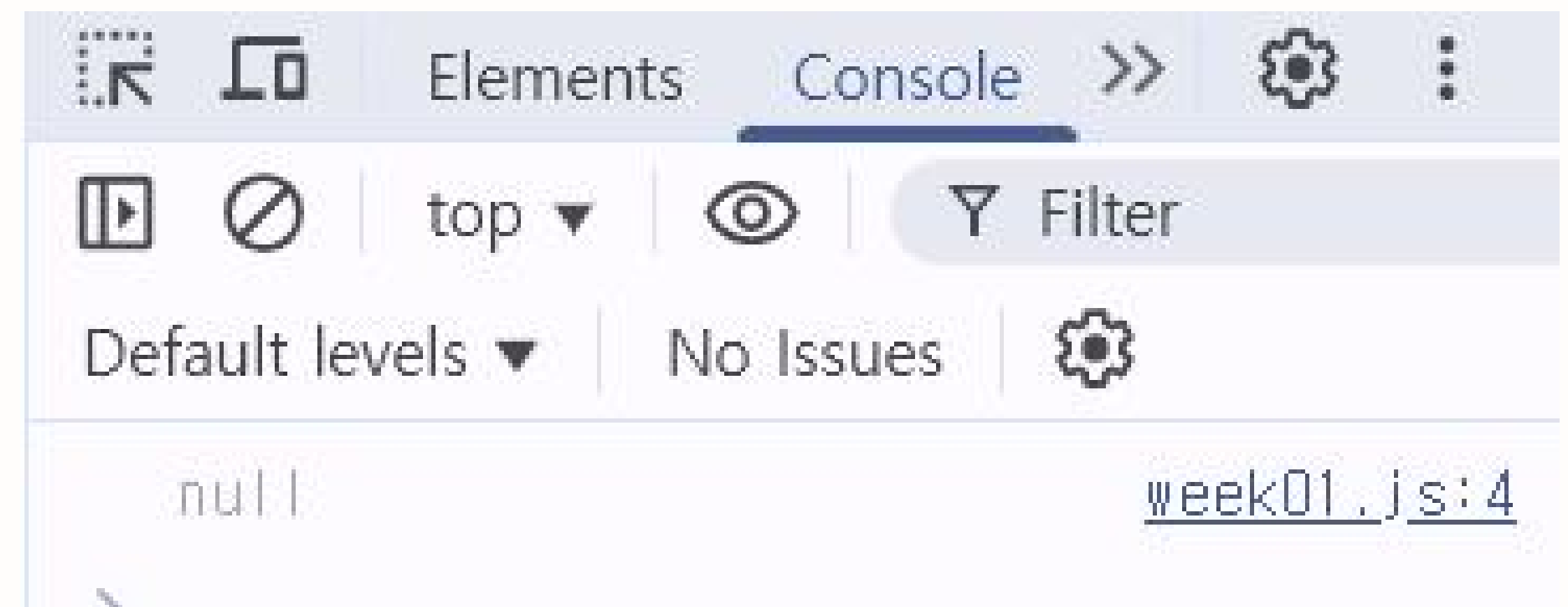
데이터 타입

null 타입

변수에 값이 없다는 것을 의도적으로 명시할 때 사용

변수에 null을 할당하는 것은 변수가 이전에 참조하던 값을 더 이상 참조하지 않겠다는 의미이다.
함수가 유효한 값을 반환할 수 없는 경우 명시적으로 null을 반환하기도 한다.

```
JS week01.js X
frontWeek01 > JS week01.js > ...
1 let likelion = "likelion";
2 likelion = null;
3
4 console.log(likelion);
```



데이터 타입

array 배열

const 변수명 = [원소1, 원소2, 원소3, 원소4, 원소5 ...]

✅ array에 항목 추가하기

배열이름.push(추가할 것)

```
JS week01.js X
frontWeek01 > JS week01.js > ...
1  const daysOfWeek = ["mon", "tue", "wed", "thu", "fri", "sat"];
2  console.log(daysOfWeek);
3
4  daysOfWeek.push("sun");
5  console.log(daysOfWeek);
```



데이터 타입

object 객체

객체 선언 시 {} 문자 안에 키 값: value 형태로 넣어주면 된다!

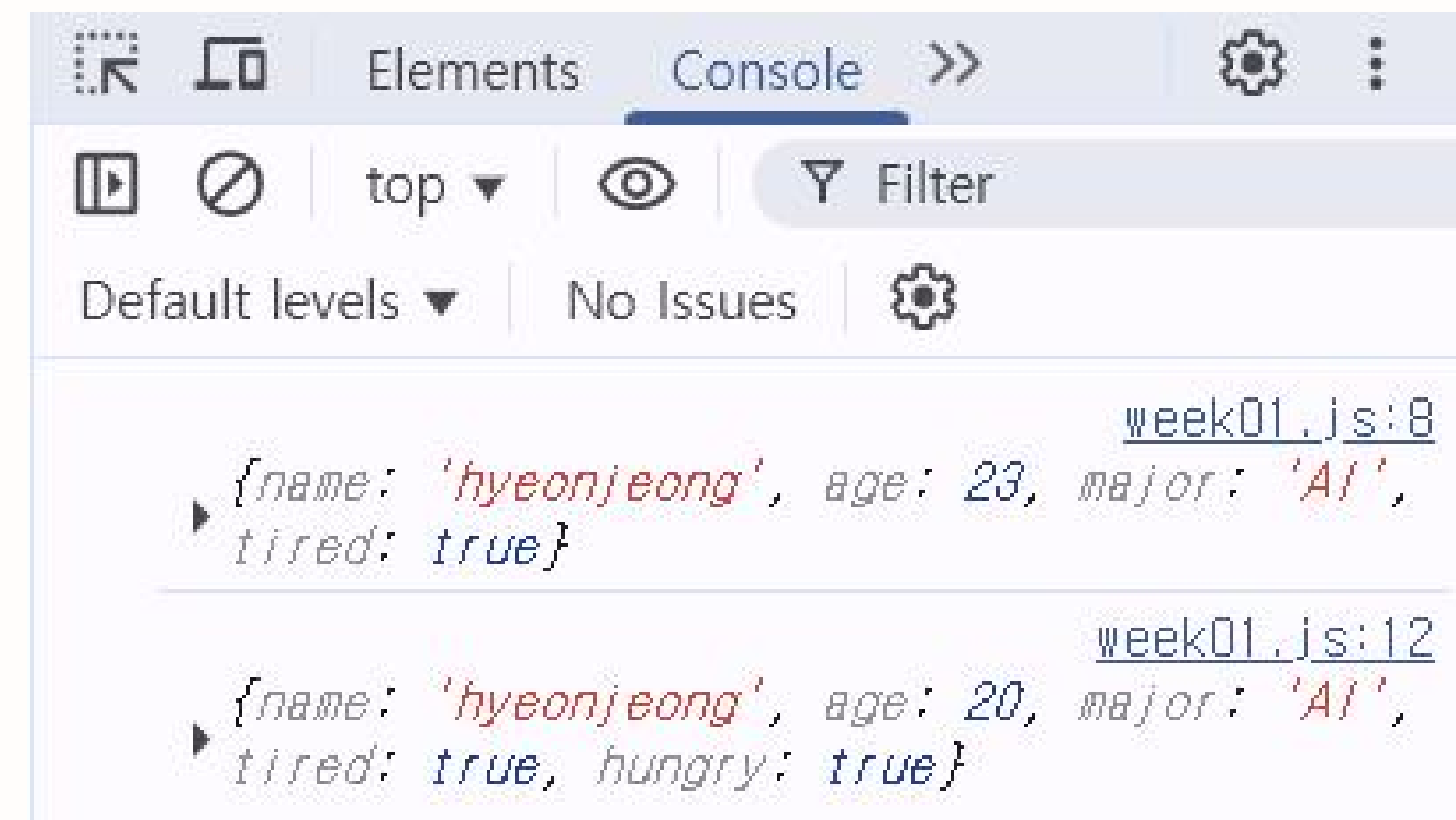
✅ 객체 value 값 참조, 수정, 삽입 시 객체이름.key 이용

frontWeek01 > JS week01.js > ...

```
1  const my = {  
2    name: "hyeonjeong",  
3    age: 23,  
4    major: "AI",  
5    tired: true,  
6  };  
7  
8  console.log(my);  
9  my.age = 20;  
10 my.hungry = true;  
11  
12 console.log(my);
```

1. Object 속성 업데이트

2. Object 속성 추가



04

연산자 및 조건문

연산자 및 조건문

자바스크립트 연산자

1. 산술 연산자: $+$, $-$, $*$, $/$, $\%$ (이항 산술 연산자) $++$, $--$ (단항 산술 연산자), $+$ (문자열 연결 연산자)
2. 할당 연산자: $=$, $+=$, $-=$, $*=$, $/=$, $\%=$
3. 비교 연산자: $==$, $===$, $!=$, $!==$ (동등/일치 비교 연산자) $>$, $<$, $>=$, $<=$ (대소 관계 비교 연산자)
4. 삼항 조건 연산자: 조건 ? 조건식이 true일 때 반환할 값 : 조건식이 false일 때 반환할 값
5. 논리 연산자: $||$ (OR), $\&\&$ (AND), $!$ (NOT)
6. typeof 연산자: typeof (피연산자) $\rightarrow$ 피연산자의 데이터 타입을 문자열로 반환

POSSIBILITY
TO
REALITY

연산자 및 조건문
비교 연산자



VS



< 동등 비교 >

자료형을 보지 않고 두 값이 일치하는지 비교

< 일치 비교 >

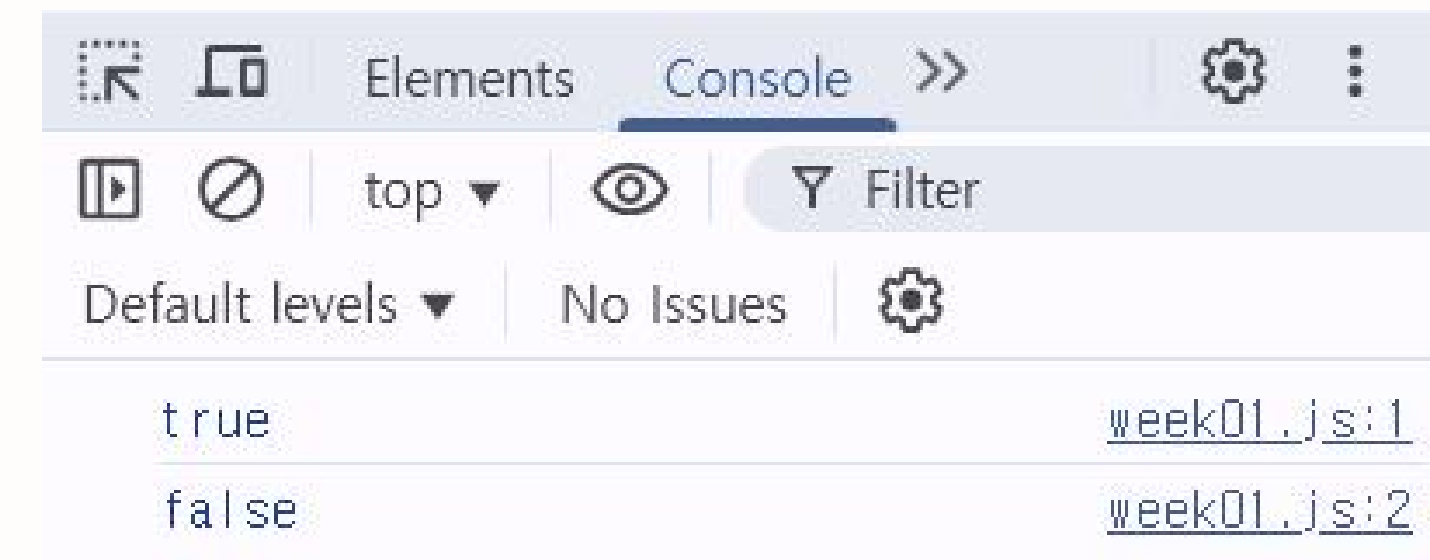
자료형을 보고 두 값이 일치하는지 비교

연산자 및 조건문

비교 연산자

```
JS week01.js X
frontWeek01 > JS week01.js
1 console.log(5 == "5");
2 console.log(5 === "5");
```

POSSIBILITY



REALITY

==

VS

===

< 동등 비교 >

자료형을 보지 않고 두 값이 일치하는지 비교

< 일치 비교 >

자료형을 보고 두 값이 일치하는지 비교

연산자 및 조건문

삼항 조건 연산자

조건 ? 조건식이 true일 때 반환할 값 : 조건식이 false일 때 반환할 값

if ... else 문을 사용하는 것과 기능은 유사하다.

삼항 조건 연산자 표현식은 값으로 평가할 수 있는 표현식인 문이다.

JS week01.js X

frontWeek01 > JS week01.js > ...

```
1 let x = 10;
```

```
2
```

```
3 let result = x % 2 ? "홀수" : "짝수";
```

```
4 console.log(result); // 짝수
```

<False로 간주되는 데이터형>

- 빈 문자열 ""
- undefined
- 값이 할당되지 않은 변수
- null

연산자 및 조건문

조건문 - if...else 문

```
if (조건식 1) {  
  // 조건식1이 참이면 이 코드 블록이 실행된다.  
} else if (조건식 2) {  
  // 조건식2가 참이면 이 코드 블록이 실행된다.  
} else {  
  // 조건식1과 조건식2가 모두 거짓이면 이 코드 블록이 실행된다.  
}
```

JS week01.js X

frontWeek01 > JS week01.js

```
1 id = prompt("아이디를 입력해주세요.");  
2 password = prompt("비밀번호를 입력해주세요.");  
3 if (id == "likelion" && password == "1234") {  
4   alert("인증에 성공했습니다.");  
5 } else {  
6   alert("인증에 실패했습니다.");  
7 }
```

5/프론트/frontWeek01/index.html

이 페이지 내용:
비밀번호를 입력해주세요.

123

확인

취소

비밀번호를 잘못 입력하면?
조건식의 값이 false가 되어
else문으로 넘어가 인증에 실패함!

이 페이지 내용:
인증에 실패했습니다.

확인

연산자 및 조건문

조건문 - switch 문

```
switch (표현식) {  
    case 표현식 1:  
        // switch문의 표현식과 표현식1이 일치하면 실행될 문;  
        break;  
    case 표현식 2:  
        // switch문의 표현식과 표현식2가 일치하면 실행될 문;  
        break;  
    default:  
        // switch 문의 표현식과 일치하는 case 문이 없을 때 실행될 문;  
}
```

if...else문보다
더욱 **다양한 상황**에 따라
실행할 코드 블록을 결정할 때 이용한다.

break문이 없으면
case 문의 표현식과 일치하지 않더라도
실행 흐름이 다음 case 문으로
이동하기 때문에 꼭! **break 문을 써**
코드 블록에서 탈출해야 한다.

05

반복문

반복문

for문

조건식이 거짓으로 평가될 때까지 코드 블록을 반복 실행하는 문

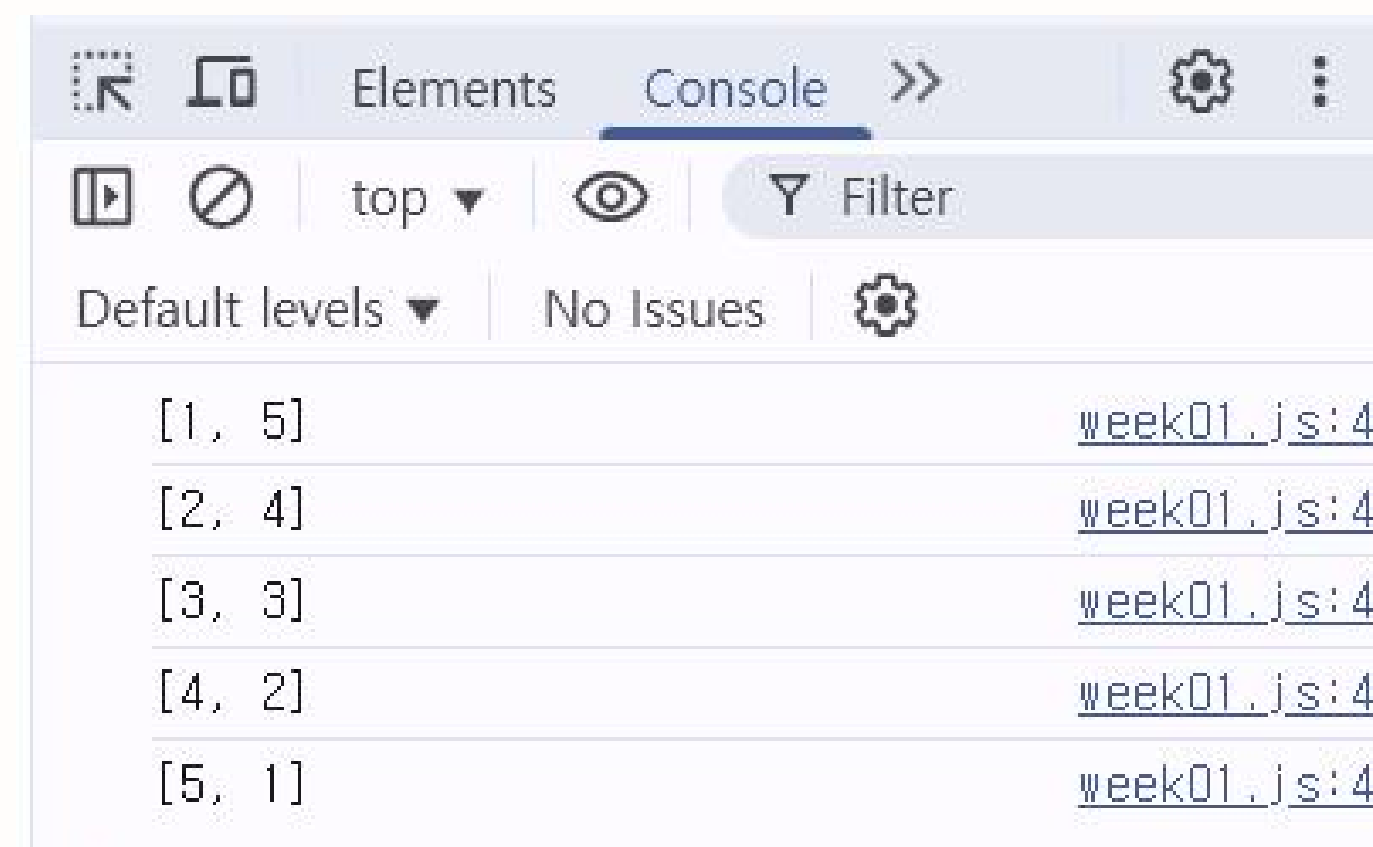
```
for (변수 선언문 또는 할당문; 조건식; 증감식) {  
    조건식이 참인 경우 반복 실행될 문;  
}
```

```
for (;;) {...} // 무한 루프
```

JS week01.js X

frontWeek01 > JS week01.js > ...

```
1 // 중첩 for문을 통한 두 수의 합이 6이 되는 모든 경우의 수 출력  
2 for (var i = 1; i <= 6; i++) {  
3     for (var j = 1; j <= 6; j++) {  
4         if (i + j === 6) console.log(`[${i}, ${j}]`);  
5     }  
6 }
```



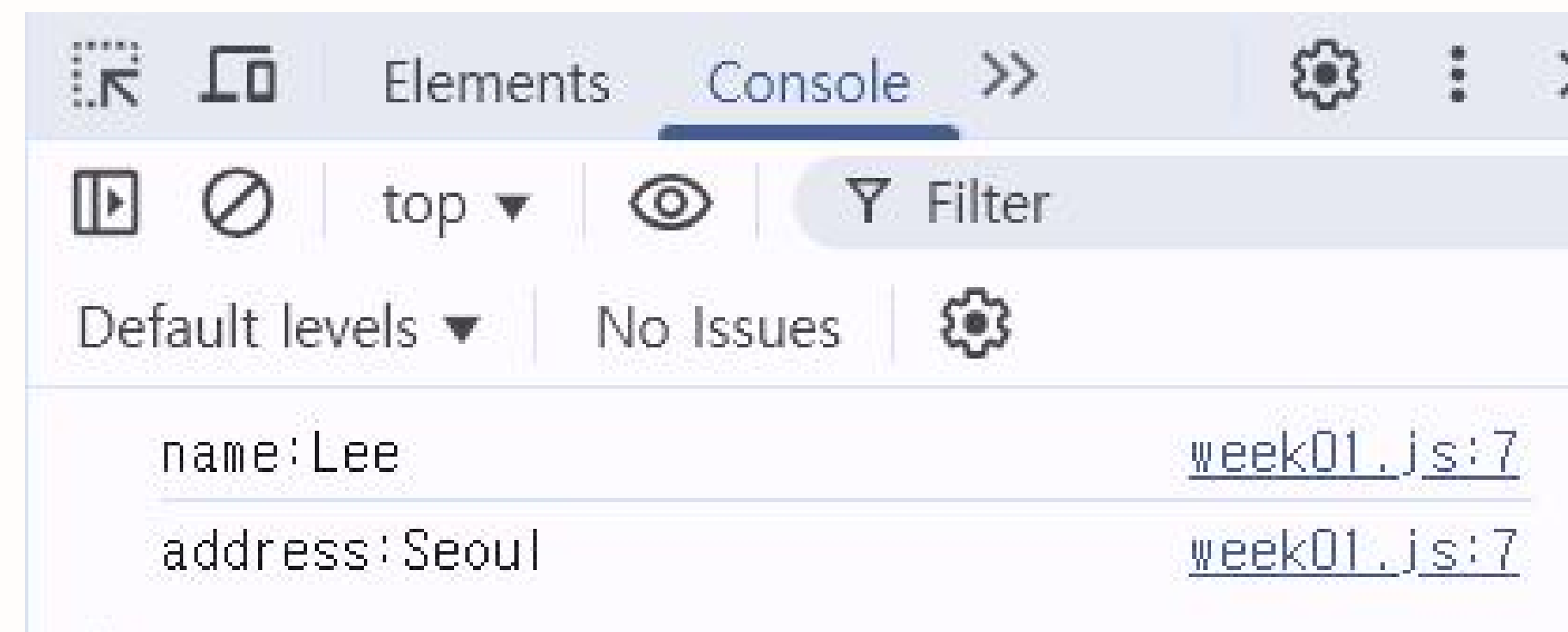
반복문

for...in 문

객체의 모든 프로퍼티를 순회하며 열거할 수 있는 반복문

for (변수 선언문 in 객체) {...}

```
JS week01.js X
frontWeek01 > JS week01.js > ...
1  const person = {
2    name: "Lee",
3    address: "Seoul",
4  };
5
6  for (const key in person) {
7    console.log(key + ":" + person[key]);
8  }
```



객체의 프로퍼티 개수만큼 순회하며,
for..in 문의 변수 선언문에서 선언한 변수에 프로퍼티 키를 할당

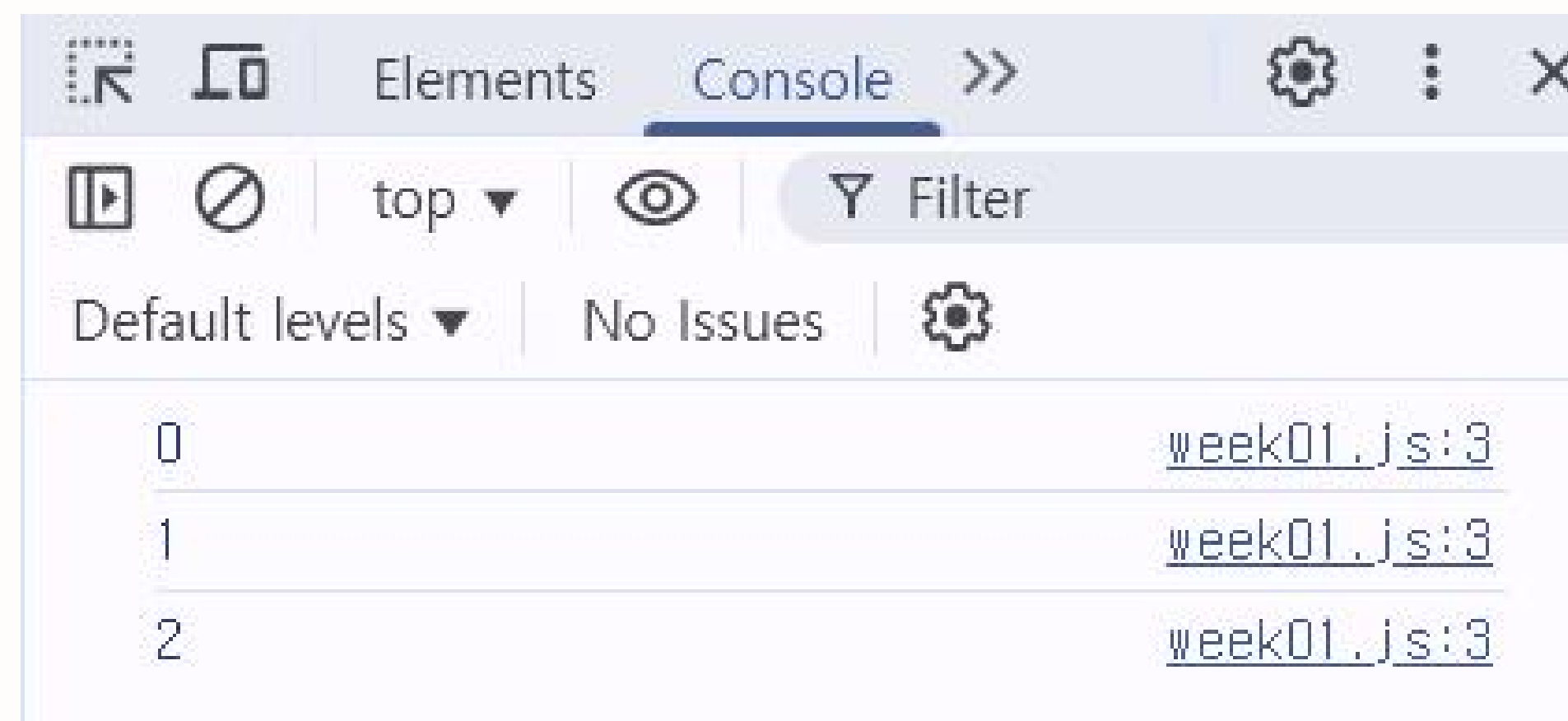
반복문

while 문 / do...while 문

while 문: 주어진 조건식의 평가 결과가 참이면 코드 블록을 계속해서 반복 실행

do...while 문: 코드 블록을 무조건 한 번 실행하고 조건식을 평가하는 반복문

```
JS week01.js X
frontWeek01 > JS week01.js > ...
1  let count = 0;
2  do {
3    console.log(count);
4    count++;
5  } while (count < 3);
```



break 문, continue 문을 통해 불필요한 반복문을 빠져나오거나
반복문의 코드 블록 실행을 현 지점에서 중단하고 반복문의 처음으로 실행 흐름을 이동시킬 수 있다!

06

함수 선언과 호출

함수 선언과 호출

자바스크립트에서 함수란?

“자바스크립트에서 함수는 값이다!”

자바스크립트에서 함수가 객체처럼 취급되기 때문에,
함수를 변수에 넣을 수 있고, 다른 함수에 인자로 줄 수도, 배열이나 객체에 넣을 수도 있다.
이러한 함수를 활용해 반복되는 코드를 효율적으로 관리할 수 있다 :)

POSSIBILITY
TO
REALITY

```
1  const sayHi = function () {  
2    console.log("안녕!");  
3  };  
4  sayHi(); // 실행됨!
```

← 함수를 변수에 넣는 경우

```
1  const actions = [sayHi, () => console.log("잘가~")];  
2  actions[1](); // 잘가~
```

← 함수를 배열에 넣는 경우

함수 선언과 호출

함수 선언

함수는 **function**으로 선언한다.

```
1 // 함수 안에 파라미터를 넣습니다. (값이 들어갈 자리)
2 function calculate(number) {
3   | console.log(((number * 10) / 2) * 3).toString());
4   | }
5                                     toString도 해당 값을 문자열로 바꾸어주는 내장함수!
6 // 실행할때는 괄호안에 아규먼트를 넣습니다. (실제 값)
7 calculate(3);
```

위 코드처럼,
함수에 **매개변수**를 넣고 싶다면 괄호 안에 설정해주고,
실제 함수 호출 시에는 괄호 안 **인자**로 값을 넣어주면 된다!

함수 선언과 호출

함수 값 반환

return을 사용해 함수 값을 반환 받는다.

```
1 function multiply1(x, y) {
2   | return x * y;
3   | }
4
5 const result1 = multiply1(2, 4);
6 console.log(result1);
```

이렇게 **return**을 통해 값을 반환 받을 경우
바로 콘솔에 출력하는 것 뿐만 아니라
변수에 함수 값을 저장해 다른 로직에 활용할 수 있다!

함수 선언과 호출

화살표 함수

화살표 함수는 기존의 함수의 표현과 내부 동작을 간략화하여 구현하는 함수이다.

```
1  function multiply1(x, y) {  
2    |   return x * y;  
3  |   }  
4  
5      function을 사용한 일반적인 함수 선언과 호출  
6  const result1 = multiply1(2, 4);  
7  console.log(result1);  
8  
9  const multiply2 = (x, y) => {  
10   |   return x * y;  
11   |   };  
12  
13      화살표 함수를 사용한 함수 선언과 호출  
14  console.log(multiply2(2, 4));
```

일반적으로 매개변수에 해당하는 값이 소괄호 안에!
매개변수를 이용한 로직이 중괄호 안에 들어간다.

당연하게도, 두 함수의 반환 결과는 같다!
상황에 따라 더 간단하고 효율적으로 쓸 수 있는
함수 선언 형태를 사용하자!

POSSIBILITY
TO
REALITY

함수 선언과 호출

Rest Parameter 문법

함수에 인자가 몇개 들어올지 모르는 상황에서 주로 사용하고, 전달된 인자들이 배열로 묶인다. 배열로 묶인 인자를 이용하여 .map이나 .forEach와 같은 메서드를 사용하기도 한다.

```
1  const multiplyAll = (...args) => {  
2    |   return args.reduce((a, b) => a * b, 1);  
3    |  
4    |   };  
5  console.log(multiplyAll(3, 4, 5)); // 60
```

1. **multiplyAll**에 들어온 인자를 모두 배열로 묶는다. 만약, 3, 4, 5가 들어온다면, **args**는 [3, 4, 5] 배열이 되는 것이다.
2. **reduce**는 배열의 요소들을 왼쪽부터 오른쪽으로 곱하는 **내장 메서드**이다. 초기 값은 1이다.
3. 즉, **Rest Parameter** 문법을 통해 배열로 변환된 매개변수들을 모두 곱한 값이 반환된다.

함수 선언과 호출

forEach

forEach 메서드는 자신의 내부에서 반복문을 실행한다.

즉, 내부에서 반복문을 통해 자신을 호출한 배열을 순회하면서, 수행해야 할 처리를 콜백 함수로 전달받아 반복 호출한다.

```
1  const numbers = [1, 2, 3];
2  const pows = [];
3
4  numbers.forEach((item) => pows.push(item ** 2));
5  console.log(pows); // [1, 4, 9]
6
```

- numbers 배열을 forEach 메서드가 순회하며 각 요소를 차례로 item에 담는다.
- 각 요소마다 거듭제곱이 적용되어 pows 배열에 추가된다.

배열을 순회하면서 각 요소에 대해 로직을 수행할 수 있게 해주는 메서드 정도로 알아두자!

함수 선언과 호출

자바스크립트 내장 함수 - Math 메서드

표준 빌트인 객체인 Math는 수학적인 상수와 함수를 위한 프로퍼티와 메서드를 제공한다.

Math.abs(num): num의 절댓값을 반환

Math.round(num): num의 소수점 이하를 반올림한 정수를 반환

Math.ceil(num): num의 소수점 이하를 올림한 정수를 반환

Math.floor(num): num의 소수점 이하를 내림한 정수를 반환

Math.sqrt(num): num의 제곱근을 반환

Math.random(num): 0에서 1 미만의 난수(랜덤 숫자)를 반환

Math.pow(a, b): a^b 값을 반환

Math.max(a,b,c ..): 전달 받은 인수 중 가장 큰 수를 반환

Math.min(a,b,c ..): 전달 받은 인수 중 가장 작은 수를 반환

07

실습 - 나만의 운세 뽑기 프로그램 만들기

실습 - 나만의 운세 뽑기 프로그램 만들기

요구사항

변수 선언 / 데이터 타입

- 프론트 기사자들의 이름을 요소로 갖는 배열을 만들어 보세요!
- 오늘의 운세 내용이 담긴 문자열 배열을 만들어 보세요!

조건문

- 각 사람의 점수에 따라 다른 코멘트를 주기 위해 if...else 조건문을 사용해 보세요!
- **hidden 기능**

반복문

- 각 사람마다 오늘의 운세와 점수를 부여할 때, forEach 반복문을 사용해 보세요!

함수

- random 관련 내장함수를 통해 오늘의 운세가 각 사람마다 무작위로 뽑히는 함수를 만들어 보세요!
- 점수에 따라 반환되는 코멘트가 달라지도록 함수를 만들어 보세요!
- 만든 함수와 데이터를 모두 사용해, 사람 별 무작위 점수에 따른 코멘트, 점수, 오늘의 운세를 콘솔에 출력해 보세요!

POSSIBILITY
TO
REALITY

실습 - 나만의 운세 뽑기 프로그램 만들기

Git Settings

1. `git pull origin main`
2. `git checkout -b "front_week01"`
3. 이번 주 실습을 위한 폴더를 하나 만들어주세요!
4. `generateFortune.js` 생성

실습 - 나만의 운세 뽑기 프로그램 만들기

변수 생성하기

```
1  const names = ["세은", "소연", "승채", "연서", "채빈", "교은"];
2
3  const fortunes = [
4    "오래 미뤄둔 일이 오늘은 의외로 쉽게 해결될 수 있습니다.",
5    "실수해도 괜찮아요. 그 안에서 더 좋은 배움이 기다리고 있어요.",
6    "지나간 일은 내려놓고, 지금 눈앞의 일을 집중해보세요.",
7    "작지만 확실한 행복이 숨어 있는 하루예요. 천천히 둘러보세요.",
8    "오늘 하루는 마음 가는 대로 행동해보는 것도 괜찮아요.",
9    "물 흐르듯 자연스럽게 흘러가는 하루가 될 거예요.",
10   "무언가 새롭게 시작하기 좋은 타이밍이에요. 주저하지 마세요!",
11 ];
12
```

POSSIBILITY
TO
REALITY

1. 프론트엔드 기사자들의 이름을 요술 담은 names 배열 생성
2. 오늘의 운세 목록, 문자열 배열 fortunes 생성

배열 선언 및 초기화 시 키워드는 **const**

실습 - 나만의 운세 뽑기 프로그램 만들기

오늘의 운세 뽑기 함수, 점수에 따른 코멘트 반환 함수 생성

```
function getRandom(arr) {  
  return arr[Math.floor(Math.random() * arr.length)];  
}  
  
function getResultComment(score) {  
  if (score == 100) return "왕큰운수의 주인공 !! 오늘은 아기사자의 왕이에요!";  
  else if (score >= 90)  
    return "무야호! 당신에겐 오늘 기적이 펼쳐질지도 몰라요!";  
  else if (score >= 70)  
    return "뽀뽀 술술 풀리는 날! 자신감 있는 아기사자가 되어보세요!!!";  
  else if (score >= 50)  
    return "평범하지만 무탈한 하루! 원래 평범함이 진짜 행복함이라는거 아시죠????";  
  else if (score >= 30) return "뚝딱거리는 하루더라도 나름 괜찮은 하루.";  
  else return "오늘은 아무것도 안 해도 괜찮은 날~!!";  
}
```

2. 점수에 따른 코멘트 반환 함수

- 인자로 들어온 점수에 따라 반환되는 코멘트가 다르도록 if...else문을 설정한다.
- 점수는 100점, 90점, 70점, 50점, 30점을 기준으로 나눈다.

1. 오늘의 운세 뽑기 함수

- Math.random()은 랜덤한 소수를 생성한다.
- 소수에 배열의 길이만큼 곱하면 0부터 배열의 길이보다 작은 랜덤 소수가 된다.
- Math.floor(...) 을 통해 소수점 아래를 버림으로써 0 ~ 4 인덱스를 만든다.
- 최종적으로 나온 인덱스에 따른 배열 요소의 값이 반환된다.

실습 - 나만의 운세 뽑기 프로그램 만들기

개인별 점수, 운세 부여 / 각 함수를 이용한 최종 출력

```
29 function generateFortuneForAll() {
30   console.log("★ 오늘의 운세 결과 ★\n");
31
32   names.forEach((name) => {
33     const fortune = getRandom(fortunes);
34     const score = Math.floor(Math.random() * 100) + 1;
35
36     console.log(`이름: ${name}`);
37     console.log(`오늘의 운세: ${fortune}`);
38     console.log(`행운 점수: ${score}점`);
39     console.log(`코멘트: ${getResultComment(score)}`);
40     console.log("-----");
41   });
42 }
43
44 // 호출
45 generateFortuneForAll();
```

2. getRandom 메서드를 사용해 각 사람마다 랜덤 운세 부여

- 인자로 들어간 배열의 무작위 요소를 반환하는 getRandom 메서드 사용

3. 각 사람별 무작위 점수 부여

- $\text{Math.random()} * 100 \rightarrow 0 \sim 100$ 을 넘지 않는 소수
- 해당 소수에 내림 메서드 $\text{Math.floor}(\dots)$ 을 적용하여 점수를 1~100까지의 정수로 바꾼다.

1. forEach를 이용한 각 사람마다의 운세 및 점수 출력

- forEach는 names 배열을 순회하며 각 요소를 차례로 name에 담는다.
- name에 들어간 요소마다 각각 랜덤 운세와 점수를 설정한다.
- 이름, 운세, 점수, 코멘트 등을 콘솔에 출력한다.
- 이때 백틱을 사용하여 문자열 안에 표현식을 삽입해 더욱 효율적으로 코드를 구성할 수 있도록 한다.

실습 - 나만의 운세 뽑기 프로그램 만들기

hidden 기능 - 발표자 뽑기

```
function generateFortuneForAll() {  
  let highestScore = 0;  
  let presenter = "";  
  
  console.log("★ 오늘의 운세 결과 ★\n");  
  
  names.forEach((name) => {  
    const fortune = getRandom(fortunes);  
    const score = Math.floor(Math.random() * 100) + 1;  
  
    console.log(`이름: ${name}`);  
    console.log(`오늘의 운세: ${fortune}`);  
    console.log(`행운 점수: ${score}점`);  
    console.log(`코멘트: ${getResultComment(score)}`);  
    console.log("-----");  
  
    // 가장 높은 점수 가진 사람 찾기  
    if (score > highestScore) {  
      highestScore = score;  
      presenter = name;  
    }  
  });  
  
  // 발표자 선정 결과 출력  
  console.log(  
    `오늘의 발표자는... 🏆 ${presenter} 님입니다! (행운 점수: ${highestScore}점)`  
  );  
}  
  
// 호출  
generateFortuneForAll();
```

발표자 뽑기 기능 구현

- 가장 높은 점수와 그에 해당하는 이름을 저장할 변수를 생성한다.
- forEach가 배열을 돌며 각 요소당 로직을 적용할 때, 조건문을 이용해 가장 높은 점수와 그를 가진 사람을 변수에 저장한다.
- 모든 배열 순회가 끝난 뒤 가장 높은 점수의 사람이 정해졌다면, 발표자를 알리는 내용이 콘솔에 출력되도록 한다.

실습 - 나만의 운세 뽑기 프로그램 만들기

콘솔 확인하기

이렇게 프론트 아기사자들의 각 운세와 발표자 출력문까지 잘 보인다면 성공! 🍀

top ▾ 🔍 Filter		Default levels ▾ No Issues
이름: 소연		week01.js:38
오늘의 운세: 지나간 일은 내려놓고, 지금 눈앞의 일을 집중해보세요.		week01.js:39
행운 점수: 82점		week01.js:40
코멘트: 뭐든 술술 풀리는 날! 자신감 있는 아기사자가 되어보세요!!!		week01.js:41
-----		week01.js:42
이름: 승채		week01.js:38
오늘의 운세: 오래 미뤄둔 일이 오늘은 의외로 쉽게 해결될 수 있습니다.		week01.js:39
행운 점수: 95점		week01.js:40
코멘트: 무야호! 당신에겐 오늘 기적이 펼쳐질지도 몰라요!		week01.js:41
-----		week01.js:42
이름: 연서		week01.js:38
오늘의 운세: 오래 미뤄둔 일이 오늘은 의외로 쉽게 해결될 수 있습니다.		week01.js:39
행운 점수: 50점		week01.js:40
코멘트: 평범하지만 무탈한 하루! 원래 평범함이 진짜 행복함이라는거 아시죠???		week01.js:41
-----		week01.js:42
이름: 채빈		week01.js:38
오늘의 운세: 오늘 하루는 마음 가는 대로 행동해보는 것도 괜찮아요.		week01.js:39
행운 점수: 3점		week01.js:40
코멘트: 오늘은 아무것도 안 해도 괜찮은 날~!!		week01.js:41
-----		week01.js:42
이름: 교은		week01.js:38
오늘의 운세: 오래 미뤄둔 일이 오늘은 의외로 쉽게 해결될 수 있습니다.		week01.js:39
행운 점수: 9점		week01.js:40
코멘트: 오늘은 아무것도 안 해도 괜찮은 날~!!		week01.js:41
-----		week01.js:42
오늘의 발표자는 ... 🧑 승채 님입니다! (행운 점수: 95점)		week01.js:52
>		

실습 - 나만의 운세 뽑기 프로그램 만들기 콘솔 확인하기

이렇게 프론트 기사자들의 각 운세와
발표자 출력문까지 잘 보인다면 성공! 🍀

1. git add .

2. git commit -m "[Feat] 나만의 운세 뽑기 프로그램 구현"

3. git push

4. 깃허브에서 merge!

5. git checkout main

6. git pull origin main

이름: 소연

오늘의 운세: 지나간 일은 내려놓고, 새로운 일을 시작하는 날! 자신감 있는 기사자가 되어보세요!!!

행운 점수: 82점

코멘트: 뭐든 술술 풀리는 날!

이름: 승채

오늘의 운세: 오래 미뤄둔 일이 오늘은 의외로 쉽게 해결될 수 있습니다.

행운 점수: 95점

코멘트: 무야호! 당신에게 오늘 기적이 펼쳐질지도 몰라요!

이름: 연서

오늘의 운세: 오래 미뤄둔 일이 오늘은 의외로 쉽게 해결될 수 있습니다.

행운 점수: 50점

코멘트: 평범하지만 무탈한 하루! 원복을 기원합니다.

이름: 채빈

오늘의 운세: 오늘 하루는 마음 가는 대로 행운을 따를 수 있습니다.

행운 점수: 3점

코멘트: 오늘은 아무것도 안 해도 괜찮은 날~!!

이름: 교은

오늘의 운세: 오래 미뤄둔 일이 오늘은 의외로 쉽게 해결될 수 있습니다.

행운 점수: 9점

코멘트: 오늘은 아무것도 안 해도 괜찮은 날~!!

오늘의 발표자는... 🧑 승채 님입니다! (행운 점수: 95점)

Default levels ▾ No Issues

- [week01.js:38](#)
- [week01.js:39](#)
- [week01.js:40](#)
- [week01.js:41](#)
- [week01.js:42](#)
- [week01.js:38](#)
- [week01.js:39](#)
- [week01.js:40](#)
- [week01.js:41](#)
- [week01.js:42](#)
- [week01.js:38](#)
- [week01.js:39](#)
- [week01.js:40](#)
- [week01.js:41](#)
- [week01.js:42](#)
- [week01.js:38](#)
- [week01.js:39](#)
- [week01.js:40](#)
- [week01.js:41](#)
- [week01.js:42](#)
- [week01.js:38](#)
- [week01.js:39](#)
- [week01.js:40](#)
- [week01.js:41](#)
- [week01.js:42](#)
- [week01.js:52](#)

08

과제 안내

과제 안내

JS로 랜덤 자판기 만들기

POSSIBILITY
TO
REALITY

1. 데이터 설정

- 음료 리스트는 배열로 저장해주세요.
- 각 음료는 이름과 가격 정보가 포함된 객체 타입이어야 합니다!
- 지갑 금액을 변수로 설정해주세요.

2. 랜덤 선택 기능

- Math 메서드를 사용해서 무작위로 음료 1개를 선택해야 합니다.

3. 출력 조건

- 선택된 음료 가격이 지갑 금액보다 작거나 같을 시 →
00 음료가 나왔어요! (가격: 000원)
지갑에 남은 돈: 000원
- 선택된 음료 가격이 지갑 금액보다 더 비쌀 시 →
돈이 부족해요! 음료를 살 수 없어요!

과제 안내

JS로 랜덤 자판기 만들기

POSSIBILITY
TO
REALITY

4. 반복 조건

- 구매 성공 여부와 관계 없이 총 3번 자판기를 시도할 수 있습니다.

5. 함수 작성

- 음료 랜덤 선택 기능 / 출력 문자열 반환 기능을 함수로 작성해주세요!
- buyDrink() 함수만 호출하면 기능이 모두 동작하도록, 함수 내부에서 자판기를 구현해주세요.
- 추가적인 기능을 고안하여 함수를 작성해주셔도 됩니다!
- 이때 JS의 다양한 내장함수를 적용해보면 더욱 좋습니다.

과제 안내

JS로 랜덤 자판기 만들기

공통 세션 때와 똑같이

- ✓ Git Issue 생성
- ✓ 과제 브랜치 생성 후 과제 수행
 - ✓ 과제 push
- ✓ PR 작성 + 담당 운영진 태그
 - ✓ merge는 운영진이!
- ✓ 다음주부터 PR 리뷰가 시작되니
PR 정리 열심히!!